Part 1: 개발 환경 구축

> VSCode 프로젝트 설정 -> Python 가상환경 구성 -> 패키지 설치 -> Streamlit 첫 실행

---

1. 프로젝트 폴더 및 VSCode 설정

1-1. 프로젝트 폴더 생성 및 열기

방법 1: 윈도우 탐색기에서 폴더 생성

1. 바탕화면의 빅데이터분석프로젝트 폴더를 더블 클릭하여 연다
2. 탐색기 안에서 **마우스 우클릭** > **새로 만들기** > **폴더** 선택
3. 폴더 이름을 아래와 같이 구분하여 입력합니다.
   - A반 : `bigdata-project-a-*****` 로 입력 ("*****"에 본인이 만들고 싶은 이름)
   - B반 : `bigdata-project-b-*****` 로 입력 ("*****"에 본인이 만들고 싶은 이름)

방법 2: 터미널에서 폴더 생성

```
mkdir bigdata-project
cd bigdata-project
```

VSCode 실행 하고 **File > Open Folder** (파일 > 폴더 열기)로 `bigdata-project-a-*` 폴더를 연다.

1-2. 필수 Extensions 확인

이미 설치되어 있을 수 있지만, 아래 항목이 있는지 확인한다:

Extension	용도
Python (Microsoft)	Python IntelliSense, 디버깅
Jupyter (Microsoft)	.ipynb 노트북 실행

1-3. 터미널 열기

`Ctrl + ` (백틱)` 로 VSCode 터미널을 연다.

또는 상단 메뉴에서 **View > Terminal** (보기 > 터미널)을 클릭해도 된다.

이번 학기 실습은 모두 **VSCode 터미널**에서 진행한다.

---

2. Python 가상환경 설정

2-1. 가상환경이란?

프로젝트마다 독립된 Python 패키지 공간을 만드는 것이다.

venv(Virtual Environment)는 파이썬의 **가상 환경**을 만들어주는 도구입니다.

```
[전역 Python] ── 모든 프로젝트가 같은 패키지를 공유 (충돌 위험)

[가상환경 방식]
├── 프로젝트A/venv ── pandas 1.5, streamlit 1.30
├── 프로젝트B/venv ── pandas 2.0, streamlit 1.42
└── 이번 수업/venv ── 우리만의 독립 환경
```

- 전역 설치 시 프로젝트 간 패키지 버전 충돌 발생 (외부 패키지의 경우 최적화 버전이 다름)
- 가상환경으로 프로젝트별 패키지를 격리하여 관리

venv vs conda 비교

구분	venv	conda (Anaconda/Miniconda)
설치	Python 내장 (별도 설치 불필요)	Anaconda/Miniconda 별도 설치 필요
Python 버전	시스템에 설치된 버전만 사용	환경별로 다른 Python 버전 설치 가능
패키지 관리	`pip` 사용	`conda` 또는 `pip` 사용
용량	가볍다	Anaconda 기준 수 GB로 무겁다
적합한 경우	일반 Python 프로젝트	데이터 과학, 복잡한 의존성 관리

> 이번 수업에서는 가볍고 별도 설치가 필요 없는 **venv**를 사용한다.

2-2. Python 버전 확인

가상환경을 만들기 전에 설치된 Python 버전을 확인한다:

```
python --version
```

> **권장 버전: Python 3.12**
>
> 이번 학기에는 **Streamlit 1.55.0(최신 버전)**을 사용할 예정이며, 이 버전은 Python 3.9~3.13을 지원한다. Python 3.13도 Streamlit에서 동작하지만, 일부 서드파티 패키지는 아직 최신 버전에서 문제가 보고될 수 있다. 이번 수업에서는 환경 통일과 호환성 검증을 위해 **Python 3.12** 사용을 권장한다.
>
> Python 3.12가 설치되어 있지 않다면 https://www.python.org/downloads/windows/ 에서 다운로드한다. (학교 시스템은 설치된 파이썬 버전을 확인 후 사용)

2-3. 실습: 가상환경 생성 및 활성화

Step 1: 가상환경 생성

```
# Python 3.12 버전을 지정하여 가상환경 생성 (권장)
py -3.12 -m venv venv
```

```
# 또는 기본 Python 버전으로 생성
python -m venv venv
```

- `py -3.12` : Windows Python 런처로 3.12 버전을 지정하여 실행
- `-m venv` : Python 내장 venv 모듈 실행
- 마지막 `venv` : 생성될 폴더 이름 (관례적으로 `venv` 사용)
- 실행 후 프로젝트 안에 `venv/` 폴더가 생성된다

> Python 3.12가 설치되어 있지 않으면 `Python 3.12 not found!` 오류가 발생한다. `py --list` 명령으로 설치된 Python 버전을 확인할 수 있다.

venv 폴더 구조

생성된 `venv/` 폴더에는 가상환경에 필요한 모든 것이 들어있다:

```
venv/
├── Scripts/            # Python 실행파일, pip, 활성화 스크립트
│   ├── python.exe      # 이 가상환경 전용 Python
│   ├── pip.exe         # 이 가상환경 전용 pip
│   └── Activate.ps1    # 활성화 스크립트
├── Lib/
│   └── site-packages/  # 이 가상환경에 설치된 패키지들 (pandas, streamlit 등)
└── pyvenv.cfg          # Python 버전 등 환경 설정 정보
```

가상환경을 활성화하면 시스템 Python 대신 `venv/Scripts/python.exe`가 사용되고, 패키지도 `venv/Lib/site-packages/`에 격리되어 설치된다. 그래서 프로젝트마다 독립된 환경이 가능하다.

Step 2: 가상환경 활성화

```
# Windows (PowerShell) - VSCode 기본 터미널
venv\Scripts\Activate.ps1

# Windows (CMD)
venv\Scripts\activate

# Windows (Git Bash)
source venv/Scripts/activate
```

활성화 성공 시 프롬프트 앞에 `(venv)`가 표시된다:

```
(venv) PS C:\Users\...\bigdata-project>
```

PowerShell 실행 정책 오류 발생 시:

>

```
.\venv\Scripts\Activate.ps1 : 이 시스템에서 스크립트를 실행할 수 없으므로
C:\Users\bigdata_jslee\venv\Scripts\Activate.ps1 파일을 로드할 수 없습니다.
자세한 내용은 about_Execution_Policies(https://go.microsoft.com/fw link/?
LinkID=135170)를 참조하십시오.
위치 줄:1 문자:1 + .\venv\Scripts\Activate.ps1 + ~~~~~~~~~~~~~~~~~~~~~~~~~~~
+ CategoryInfo :
보안 오류: (:) [], PSSecurityException + FullyQualifiedErrorId :
UnauthorizedAccess
```

이 경우 관리자 권한 PowerShell에서 아래 명령어를 한 번 실행하세요:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

이 설정은 외부에서 생성된 스크립트(가상환경 활성화 파일 등)를 현재 사용자 환경에서 실행할 수 있도록 허용하는 보안 규칙 변경이다. 한 번만 실행하면 이후에는 다시 설정할 필요 없다.

위 명령 실행 후 다시 활성화를 시도한다.

---

3. 패키지 설치 및 관리

3-1. 이번 학기 핵심 패키지 설치

```
pip install pandas streamlit matplotlib seaborn requests scikit-learn
```

패키지	용도
`pandas`	데이터프레임 기반 데이터 분석
`streamlit`	웹 대시보드/앱 제작
`matplotlib`	기본 시각화
`seaborn`	통계 시각화
`requests`	API 호출, 데이터 수집
`scikit-learn`	머신러닝 모델 학습/평가

matplotlib/seaborn은 왜 필요한가?

Streamlit에도 `st.bar_chart()`, `st.line_chart()` 같은 내장 차트가 있지만, 간단한 차트(막대, 선, 영역)만 지원하고 혼동행렬 히트맵, 박스플롯, 분포도 등은 지원하지 않는다. 세밀한 스타일 커스터마이징도 어렵다.

ML 프로젝트에서 필수적인 시각화(혼동행렬, ROC 커브 등)는 matplotlib/seaborn으로 그리고, Streamlit에서 `st.pyplot()`으로 해당 차트를 웹 앱에 표시할 수 있다.

> **Tip: 한글 폰트 깨짐 방지**
>
> Matplotlib 차트에서 한글이 깨진다면 코드 상단에 아래 설정을 추가한다:
>
> ```
> import matplotlib.pyplot as plt
> plt.rc('font', family='Malgun Gothic')  # Windows 기준
> plt.rcParams['axes.unicode_minus'] = False  # 마이너스 기호 깨짐 방지
> ```

3-2. 설치 확인

```
pip list
```

출력 예시:

```
Package         Version
--------------- --------
pandas          2.2.x
streamlit       1.42.x
matplotlib      3.9.x
seaborn         0.13.x
requests        2.32.x
scikit-learn    1.6.x
...
```

3-3. requirements.txt 생성

```
pip freeze > requirements.txt
```

- 설치된 모든 패키지와 버전을 기록하는 파일
- 다른 PC나 팀원이 동일 환경을 구성할 때 사용:

```
pip install -r requirements.txt
```

---

4. Streamlit 첫 실행

4-1. Streamlit이란?

Streamlit에 대한 설명은 다음 수업에서 상세히 진행 합니다.

- Python만으로 웹 앱을 만들 수 있는 프레임워크
- HTML/CSS/JS 없이 Python 코드만으로 대시보드 구현 가능

- 이번 학기 모든 프로젝트의 결과물을 Streamlit으로 배포한다

공식 문서: https://docs.streamlit.io/

4-2. 실습: 첫 번째 Streamlit 앱

프로젝트 폴더에 `app.py` 파일을 생성하고 아래 코드를 작성한다:

```
import streamlit as st
import pandas as pd

# 페이지 설정
st.set_page_config(page_title="빅데이터 분석 프로젝트", page_icon="📊")

# 제목
st.title("빅데이터 분석 프로젝트")
st.write("첫 번째 Streamlit 앱입니다!")

# 구분선
st.divider()

# 간단한 데이터프레임 표시
st.subheader("샘플 데이터")
data = {
    "이름": ["김철수", "이영희", "박민수", "정수진", "최지훈"],
    "학년": [3, 3, 3, 3, 3],
    "전공": ["AI소프트웨어", "AI소프트웨어", "AI소프트웨어", "AI소프트웨어", "AI소프트
웨어"],
    "Python점수": [85, 92, 78, 95, 88]
}
df = pd.DataFrame(data)
st.dataframe(df, use_container_width=True)

# 간단한 차트
st.subheader("Python 점수 차트")
st.bar_chart(df.set_index("이름")["Python점수"])

# 사이드바
st.sidebar.header("설정")
st.sidebar.write("이 영역은 사이드바입니다.")
name = st.sidebar.text_input("이름을 입력하세요")
if name:
    st.sidebar.write(f"안녕하세요, {name}님!")
```

4-3. Streamlit 실행

```
streamlit run app.py
```

만약 위 명령어에서 오류 발생 시 아래 시도

```
python -m streamlit run app.py
```

실행 결과:

```
  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.x.x:8501
```

- 브라우저가 자동으로 열리며 앱이 표시된다
- 코드를 수정하고 저장(`Ctrl + S` 또는 **File > Save**)하면 브라우저에서 바로 반영된다

4-4. Streamlit 주요 명령 정리

함수	기능
`st.title()`	페이지 제목
`st.subheader()`	소제목
`st.write()`	텍스트, 데이터프레임 등 범용 출력
`st.dataframe()`	인터랙티브 데이터프레임 표시
`st.bar_chart()`	막대 차트
`st.sidebar`	사이드바 영역
`st.text_input()`	텍스트 입력 위젯
`st.divider()`	구분선

4-5. Streamlit 종료

터미널에서 `Ctrl + C`를 누르면 Streamlit 서버가 종료된다.

---

5. Git/GitHub 저장소 연결 (실습 마무리)

프로젝트를 GitHub에 올려서 이번 Part 1 실습을 마무리한다.

```
# 1. .gitignore 생성 (venv 폴더 제외)
echo "venv/" > .gitignore
echo "__pycache__/" >> .gitignore
echo ".env" >> .gitignore

# 2. Git 초기화 및 커밋
git init
git add .
```

```
git commit -m "2주차: 개발환경 구축 및 Streamlit 첫 실행"

# 3. GitHub 원격 저장소 연결 및 Push
git branch -M main
git remote add origin https://github.com/본인계정/bigdata-project.git
git push -u origin main
```

> **GitHub 인증 안내:** `git push` 실행 시 GitHub 인증이 필요하다. 처음 push하면 브라우저 팝업으로 GitHub 로그인 창이 뜨거나, 터미널에서 인증을 요청한다. Windows에 Git 설치 시 함께 설치되는 **Git Credential Manager**가 있으면 한 번 로그인 후 인증 정보가 자동 저장되어 이후에는 별도 로그인 없이 push할 수 있다.

> Git/GitHub 사용법이 기억나지 않으면 부록 A를 참고한다.

체크포인트

- ☐ VSCode에서 `bigdata-project` 폴더가 열려 있다
- ☐ 가상환경이 활성화되어 터미널에 `(venv)`가 표시된다
- ☐ `pip list`에 pandas, streamlit, matplotlib, seaborn, requests, scikit-learn이 있다
- ☐ `requirements.txt` 파일이 생성되어 있다
- ☐ `streamlit run app.py`로 브라우저에서 앱이 정상 표시된다
- ☐ GitHub에 코드가 push되어 있다

부록 A: Git/GitHub 가이드

> 수업 시간에는 다루지 않습니다. 필요 시 참고하세요.

A-1. Git이란?

- **Git**: 로컬에서 동작하는 버전 관리 프로그램
- **GitHub**: Git 저장소를 온라인에 호스팅하는 서비스

```
[로컬 PC]                    [GitHub (원격)]
작업 디렉토리                  원격 저장소
    |                            ^
    v git add                    |
스테이징 영역                   | git push
    |                            |
    v git commit                 |
로컬 저장소 ---------------------+
```

A-2. Git 설치 확인

```
git --version
```

미설치 시: https://git-scm.com/ 에서 다운로드

A-3. Git 최초 설정 (처음 한 번만)

```
git config --global user.name "본인이름"
git config --global user.email "본인이메일@example.com"
```

A-4. 로컬 저장소 초기화 및 첫 커밋

```
# Git 저장소 초기화
git init

# .gitignore 파일 생성
```

`.gitignore`는 Git이 **추적하지 않을 파일/폴더**를 지정하는 파일이다. 아래 항목들은 코드가 아니거나, 개인 환경에 따라 달라지는 파일이므로 GitHub에 올릴 필요가 없다.

`.gitignore` 내용:

```
# 가상환경
venv/

# Python 캐시
__pycache__/
*.pyc

# 환경 변수
.env

# IDE 설정
.vscode/
.idea/

# OS 파일
.DS_Store
Thumbs.db
```

항목	제외하는 이유
`venv/`	가상환경 폴더는 용량이 크고, `requirements.txt`로 재생성 가능
`__pycache__/`, `*.pyc`	Python 실행 시 자동 생성되는 캐시 파일
`.env`	API 키, 비밀번호 등 민감한 환경 변수 파일

항목	제외하는 이유
`.vscode/`, `.idea/`	개인 IDE 설정 (사람마다 다름)
`.DS_Store`, `Thumbs.db`	macOS/Windows가 자동 생성하는 시스템 파일

```
# 파일 스테이징 및 커밋
git add .
git commit -m "프로젝트 초기 설정"
```

A-5. GitHub 원격 저장소 생성 및 연결

1. https://github.com 로그인
2. 우측 상단 `+` > `New repository` 클릭
3. Repository name: `bigdata-project`
4. **Public** 선택, `Add a README file` 체크 **해제**
5. `Create repository` 클릭

```
# 원격 저장소 연결
git remote add origin https://github.com/본인계정/bigdata-project.git

# 브랜치 이름 변경 및 Push
git branch -M main
git push -u origin main
```

A-6. Private 저장소 접근 (Access Token 사용)

GitHub Private 저장소를 clone하거나 push할 때 인증이 안 되는 경우, **Personal Access Token**을 사용하여 접근할 수 있다.

Step 1: Access Token 발급

1. GitHub 로그인 > 우측 상단 프로필 > **Settings**
2. 좌측 메뉴 하단 **Developer settings** > **Personal access tokens** > **Tokens (classic)**
3. **Generate new token (classic)** 클릭
4. Note에 용도 입력 (예: `bigdata-project`), Expiration 설정
5. **repo** 권한 체크 > **Generate token** 클릭
6. 생성된 토큰(`ghp_...`)을 복사하여 안전한 곳에 저장 (다시 확인 불가)

Step 2: Token으로 clone

```
git clone https://<ACCESS_TOKEN>@github.com/계정명/저장소명.git
```

예시:

```
git clone https://ghp_abc123@github.com/ljs535-source/BigDataAnalysis.git
```

Step 3: 이미 clone한 저장소에 Token 적용

```
git remote set-url origin https://<ACCESS_TOKEN>@github.com/계정명/저장소명.git
```

> **주의:** Access Token은 비밀번호와 같으므로 절대 코드에 직접 포함하거나 GitHub에 올리지 않는다. `.env` 파일에 저장하고, `.gitignore`에 `.env`를 추가하여 관리한다.

A-7. 이후 작업 흐름 (반복)

```
git add .
git commit -m "변경 내용 설명"
git push
```

A-8. 유용한 Git 명령어

명령어	설명
`git status`	현재 변경 상태 확인
`git log --oneline`	커밋 이력 간략 조회
`git remote -v`	연결된 원격 저장소 확인
`git remote remove origin`	원격 저장소 연결 해제

A-9. 자주 발생하는 오류

오류 메시지	원인	해결 방법
`fatal: not a git repository`	`git init` 미실행	`git init` 실행
`error: src refspec main does not match any`	커밋 없이 push 시도	`git add .` + `git commit` 먼저 실행
`remote: Permission denied`	GitHub 인증 실패	GitHub 로그인 확인
`fatal: remote origin already exists`	origin 이미 등록됨	`git remote remove origin` 후 재추가